

Project Title: Language Translator Spanish to English

Yeshwanth Kodam
koday01@pfw.edu
dept.of computer science
PURDUE University

Chaitanya Kumar Battula
battc01@pfw.edu
dept.of computer science
PURDUE University

Neha Pathak
pathn02@pfw.edu
dept.of computer science
PURDUE University

March 2, 2023

Abstract

This research aims to develop a Spanish to English language translator model using NLP. The main aim of this research is to maintain the accuracy and the meaning of the original sentence or word.

I Introduction :

Effective communication is crucial in today's globalized world, but language barriers can often pose a challenge. This is where language translation comes in. One such translator is the Spanish to English language translator, which can convert Spanish sentences to English without losing the meaning of the original sentence. This tool can help bridge the gap between different cultures and facilitate communication and understanding.

II Motivation:

In order to break the language barrier :most of our university workers speak only Spanish, and also in general most of the people who speak Spanish won't understand other languages. Promote cross cultural understanding: By breaking down language barriers, a Spanish-to-English language translator can help to promote cross-cultural understanding. Increase efficiency and productivity: Improved communication and understanding can lead to increased efficiency and productivity.

III Related Work:

1. Google Translate: Google Translate is a popular online language translator that supports a wide range of languages, including Spanish and English. It uses machine learning algorithms and neural networks to provide accurate and fast translations.

2. DeepL Translator: DeepL Translator is a relatively new language translator that uses artificial intelligence and neural networks to provide high-quality translations. It supports several languages, including Spanish and English, and offers fast and accurate translations.

IV Methodology:

A Data Set:

We have downloaded the dataset from the link:<http://www.manythings.org/anki/spa-eng>. The dataset has bilingual English and Spanish sentences, with over 139,013 pairs aligned so that each English sentence corresponds to its Spanish translation. The data is in a TSV format with each record containing a sentence pair along with license and attribution information, which we removed as it is not relevant to the training.

B Exploratory Data Visualization:

The study performed exploratory analysis on a DataFrame containing both text data, printing its shape and reporting the number of null and duplicate values. The most popular terms in each language were visualized using WordCloud. The distribution of phrase lengths was visualized using seaborn's hist plot. The dataset was reduced to the first 60,000 pairs due to its large size. The distinct English and Spanish sentence lengths and maximum phrase lengths were also determined, with a maximum length of 10 for English and 19 for Spanish.

C Pre-Process Data:

As the data set contains both the English and Spanish text, we had to pre-process for both the text of the reduced dataset. The following pre-processing steps were performed:Removed the punctuation for

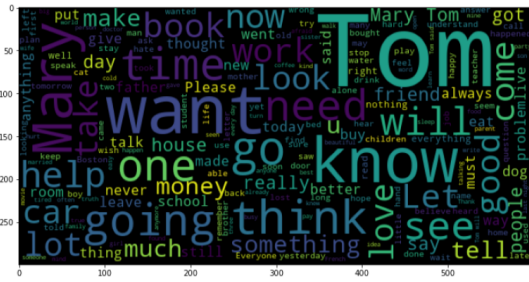


Figure1: Visualization Of The Most Common English Word.



Figure2 : Visualization Of The Most Common Spanish Word.

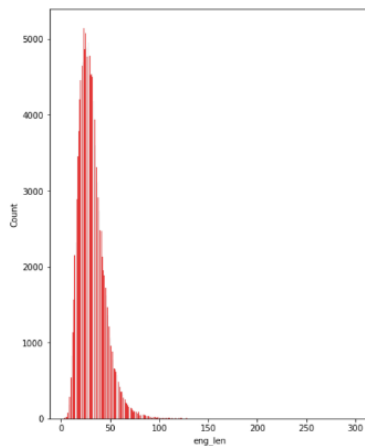


Figure 3: Distribution chart for English length for the entire dataset.

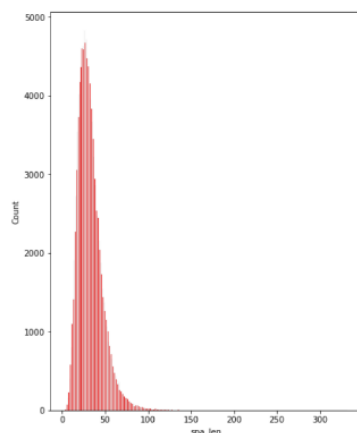


Figure 4: Distribution chart for Spanish length for the entire dataset.

most English and Spanish text. Convert both the

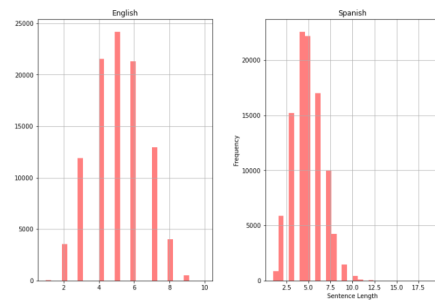


Figure 5: Distribution chart for both English and Spanish length for the reduced dataset(first 60,000 pair).

text to lower case.

D Tokenization:

The Tokenizer class from the Keras library was used to perform word-level tokenization on text data for NLP projects. The functions 'prepare English tokenizer' and 'prepare Spanish tokenizer' utilized the tokenization function to tokenize English and Spanish sentences respectively, and output the vocabulary size for each language. The English vocabulary size was found to be 10643 and the Spanish vocabulary size was found to be 21744.

E Text to Sequence Conversion:

Since we are using, a Seq2Seq model requires that we convert both the input and the output sentences into integer sequences of fixed length. We have captured the lengths of all the sentences in two separate lists for English and Spanish are 10 and 19, we got this result from the data analysis part. And we have to also generate training data and validation data.

F Split Data Set:

The train-test-split function from scikit-learn is used to generate a split dataset function that divides the clean dataset into training and test datasets. For training and testing, the split is done 80:20, respectively. The input and target sequences for training and testing the model are prepared using the obtained datasets.

G Preparing Training and Validation Data:

This function turns each text line into a series of numbers using a tokenizer object in order to prepare text data for input into a neural network. The `pad_sequences()` function from Keras is then used to pad the sequences with zeros to make them all the same length. The padded sequences are returned as a NumPy array by the function, which also accepts a tokenizer object, a list of text lines, and the maximum length of the sequences. The output consists

of fixed-length integer sequences that can be input directly into the neural network. This produces test data (Test X, Test Y) and training data (Train X, Train Y), where Train X is Spanish. Test X is a Spanish sentence, while Sentences and Train Y are English sentences for training.

V Model:

A Sequence-to-Sequence Model:

Sequence-to-sequence learning (Seq2Seq) is about training models to convert sequences from one domain (e.g. sentences in Spanish) to sequences in another domain (e.g. the same sentences translated to English). For example: translating “Qué harás hoy” from Spanish to English has input of 3 words and output of 5 symbols (What are you doing today?).

A sequence to sequence model aims to map a fixed-length input with a fixed-length output where the length of the input and output may differ. Both the input and output are sentences. In other words, these sentences are a sequence of words going in and out of a model. This is the basic idea of Sequence-to-Sequence modeling. The figure below tries to explain this method.

B LSTM Layer:

Long Short-Term Memory (LSTM) networks are a modified version of recurrent neural networks, which makes it easier to remember past data in memory. The vanishing gradient problem of RNN is resolved here. LSTM is well-suited to classify, process and predict time series given time lags of unknown duration. It trains the model by using back-propagation.

VI Experiment:

A Model Building:

In Our project we have used the seq2seq model, the typical Seq2Seq model architecture consists of an encoder and a decoder, both these parts are essentially two different recurrent neural network (RNN) models combined into one giant network. Encoder layer consists of an embedding layer and an LSTM layer. Decoder layer consists of another LSTM layer followed by a dense layer.

The input for the model integer sequences of fixed length of both TrainX(Spanish) and TrainY(English) generates an output of a sequence of integers representing words in the target language.

B Architecture:

The model consists of 5 layers: Embedding layer, two LSTM layer, RepeatVector layer, Dense layer.

Embedding Layer:

An embedding layer that takes as input the Spanish vocabulary size (11,132), the number of units to represent each word (512), and the input length (10). It outputs a sequence of embedded vectors with dimensionality (10, 512). This layer has 5,693,184 parameters. The main purpose of using this layer is to learn to vectorize each word in the input sequence.

LSTM Layer:

An LSTM (Long Short-Term Memory) layer that takes as input the embedded sequences and outputs a single vector of size 512 that captures the contextual information of the input sequence. This layer has 2,099,200 parameters. The LSTM layer is a type of recurrent neural network (RNN) that can take variable-length sequences as input and provide output as a fixed-length vector.

Repeat Vector Layer:

A Repeat Vector layer that repeats the output vector of the previous layer (size 512) 10 times, which is the length of the output sequence. This layer has no trainable parameters

LSTM layer:

Another LSTM layer that takes as input the repeated vector and generates a sequence of vectors of size (10, 512). This layer has 2,099,200 parameters.

Dense Layer:

A dense layer is used with the softmax activation function and LSTM layer output to generate a probability distribution over the English vocabulary size of 10,643. This layer has 5,459,859 parameters and maps the output vector to a probability distribution over the output vocabulary with the softmax function.

C Model Compilation:

The model is trained using RMSprop optimizer with learning rate 0.001 and sparse categorical cross-entropy loss function for 40 epochs with a batch size of 512. The model is saved using a model checkpoint callback, and training/validation accuracy and loss are plotted using matplotlib (Figure 7)

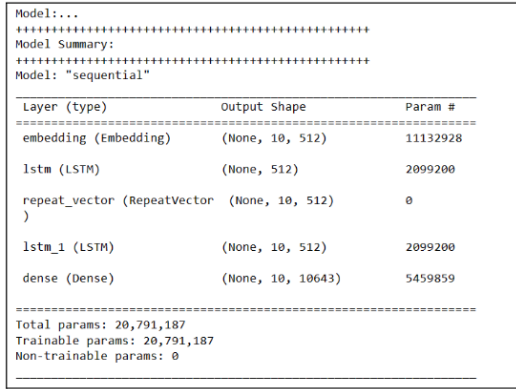


Figure 6: Model Summary of Seq2Seq Model .

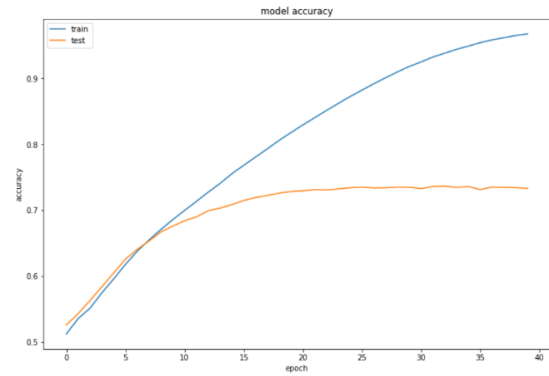


Figure 7: Model Accuracy. .

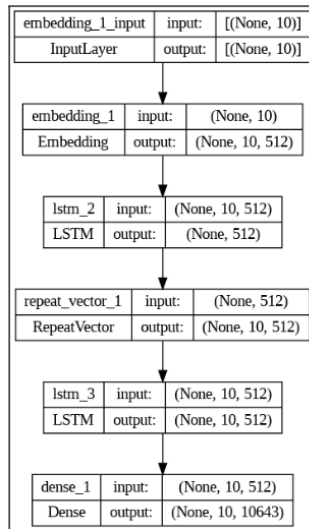


Figure 6: Model Architecture of Seq2Seq Model .

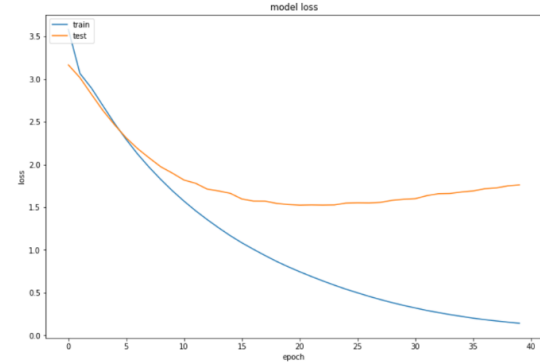


Figure 8: Model Loss, .

quences back to text by returning the word that corresponds to that word in the tokenizer's word index. Each word index is turned back into a word as the function runs over the expected sequences, combining the words with spaces to form the associated English phrase. Lastly, a list of predicted English sentences is returned.

E Model Evaluation and Results:

The evaluation metrics used in the code are accuracy and BLEU score. The accuracy measure measures the percentage of correctly predicted words in the target language compared to the actual words in the test set. The BLEU score measures the quality of the generated translation by comparing it to multiple reference translations. It is a popular metric used in machine translation and evaluates the n-gram overlap between the predicted and reference translations. The score ranges from 0 to 1, with higher values indicating better translations. So in our code, after running the model (epoch = 40).

F Additional Analysis and Discussion:

We can see in fig 7,8. The validation loss stopped decreasing after 20 epochs, and the validation accuracy has also stopped decreasing. A typical BLUE score value ranges from 0 to 1, and here the BLEU

No:	Parameters	Value
1	Learning Rate	0.001
2	LSTM Units	512
3	Activation Function	Softmax
4	Loss Function	sparse categorical cross entropy
5	Validation Split	0.2
6	Epochs	40
7	Batch Size	512

Table 1 : Training Parameters for Seq2Seq Model

D Prediction:

The function employs a trained model that is loaded from a file called "model.h1" to produce predictions for the test data. The test data is reshaped to match the input shape of the model, and predicted sequences are then obtained using the model's predict() method. The predicted sequences are returned after determining the word index with the highest predicted probability at each time step using the np.argmax() method. A function that takes an integer n and a tokenizer object converts se-

Epochs	Accuracy	Loss	Val_accuracy	val_loss	Time (in S)
1	0.512	3.582	0.526	3.16	524
5	0.59	2.48	0.60	2.46	598
10	0.68	1.69	0.67	1.89	627
15	0.75	1.16	0.75	1.66	692
20	0.81	0.80	0.72	1.53	616
25	0.87	0.54	0.73	1.54	557
30	0.91	0.32	0.73	1.69	583
35	0.95	0.22	0.73	1.69	583
40	0.96	0.14	0.73	1.76	634

Table 2: Traing Metrics

No:	Evaluation	Values
1	Accuracy	73.10 %
2	BLEU Score	0.256

Table 3: Testing Metrics

score value is 0.256, which is way below the ideal value of 0.5. Our model did a decent job. But there are several cases where it misses out on understanding of the key words. For example, it fails translates “im tired of boston” to “im am boston”.

VII Challenges:

One of the challenges we faced is Computational complexity, Model translation is a computationally complex task, and training a high-quality system requires significant computational resources. From Table 2, we can see that for model training for 1 epoch which took around 524 s , which is equal to 8.7 min. Overall for 40 epoch it took 20 h 32 min. During this phase we have faced a number of time system crashes.

Another challenge is Data availability, for accurate translation of a text the model requires a large amount of the data to learn how to translate efficiently. Since, In our project due to Computational complexity we have used 60,000 pairs , as this might not be enough to train the model, to accurately translate a text.

The language characteristics pose a challenge in the model as both English and Spanish have different grammar, vocabulary, and sentence structures. This results in cases where the predicted sentence may not exactly match the actual sentence, leading to a lower BLEU score. For instance, the actual sentence "suddenly it began to rain" may be predicted as "suddenly raining", which, though different grammatically, has the same meaning in a

human context.

VIII Future Work:

One of the major issues for future work is that, since we have used a reduced dataset, the BLEU score is less than the ideal value (0.5). In order to improve the BLEU score, it is using a GPU where it can train the model on the entire dataset. Thereby, the model will be able to learn rare words, idioms, and the right context and be able to predict the accuracy of the words. So, if we had used a GPU , our project could be used for real time applications.

Machine translation models are frequently assessed using automated measures like the BLEU score. These measurements, however, sometimes do not correlate well with human judgment. To better comprehend the benefits and drawbacks of machine translation models, future work may involve undertaking human evaluation studies.

IX Conclusion:

To conclude, the seq2seq model is used in our translator, where an encoder-decoder architecture is used, further trained the model on a Spanish-English translation dataset, and evaluated its performance using various metrics such as accuracy, loss, and BLEU score. We have also discussed the challenges faced while building the model and possible future improvements to enhance its performance. This is our GitHub link: <https://github.com/ChaitanyaKumarBattula/NLP-Semester-Project/tree/main/NLP%20Language%20Transalation>.

X Reference:

1. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin: Attention Is All You Need <https://arxiv.org/abs/1706.03762v1>
2. Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio: Neural Machine Translation by Jointly Learning to Align and Translate <https://arxiv.org/abs/1409.0473>
3. Peng Cui, Le Hu, Yuanchao Liu: Inducing Alignment Structure with Gated Graph Attention Networks for Sentence Matching <https://arxiv.org/abs/2010.07668>